

A SIMPLE GUIDE TO CHOOSING A DATABASE

MongoDB vs. Apache Cassandra

For Smart Traffic Management Systems

How two popular databases handle city traffic data

Tanisha Patel

Guided by Dr. Hemangini Patel

Department of Computer Science and Information Technology

Vanita Vishram Women's University

What We'll Cover

01

Introduction

Why traffic data is so hard to manage

02

Literature Review

15 studies we looked at, from 2016 to 2025

03

Research Gap

5 problems no one has solved yet

04

Problem Statement

The MongoDB vs. Cassandra trade-off

05

Methodology

Our 5-step research process

06

Database Design & Results

How each database is built, and how they compare

07

Hybrid Architecture

Using both databases together

08

Future Scope & Conclusion

What comes next

Why Smart Traffic Needs Better Data Tools

Cities are growing fast, and so is traffic. Sensors, cameras, and connected cars create huge amounts of data every second. Old-style databases simply can't keep up.

68%

of people will
live in cities by 2025

4.4B

car trips happen
every day, worldwide

100 TB

of traffic data made
every day, in one city

\$300B+

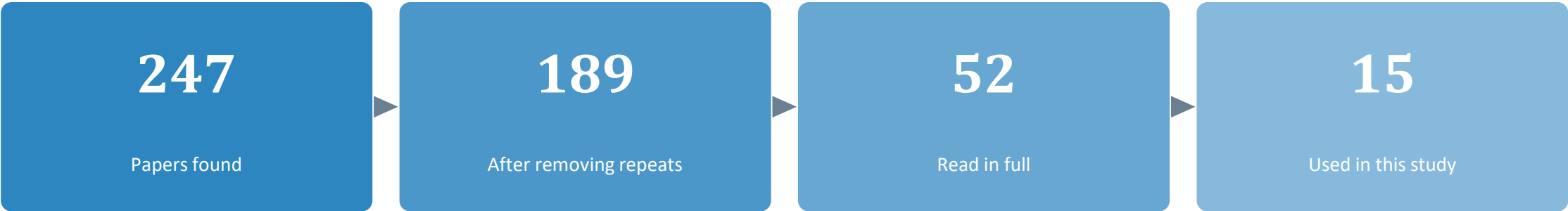
lost every year to
traffic jams (USA)

Why old databases struggle: they use fixed formats, they're hard to expand, and they slow down when too many people use them at once.

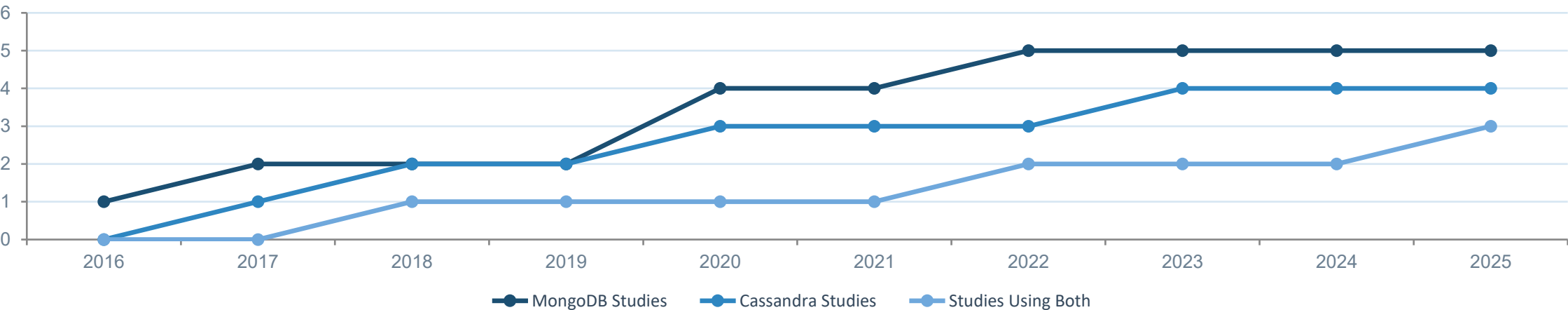
The new answer — NoSQL: MongoDB and Apache Cassandra are two popular databases built to grow easily and handle huge, fast-changing data — exactly what smart traffic systems need.

How We Studied the Research

We started with 247 papers and narrowed them down to the 15 best studies, all published between 2016 and 2025.



How Research Grew Over Time



Four Stages of Research

2016–2017

Getting Started

MongoDB was easy to set up but weak at backing up data across cities.
Cassandra was reliable but hard to search.

2018–2019

Testing & Building

Cassandra became very fast at reading recent data. Tests showed MongoDB is better for reading, Cassandra is better for writing.

2020–2021

Real-World Use

MongoDB slowed down under heavy traffic. Cassandra could switch to a backup city in under 30 seconds with no data lost.

2022–2025

Smart & Combined

Teams began using both databases together and adding AI. This combo cut costs by about 35%.

5 Problems Still Unsolved

1

No Standard Way to Test

Current tests use fake data that doesn't match real traffic patterns.

2

No Clear Safety Rules

No study shows exactly which settings keep traffic lights safe and in sync.

3

No Full Combined System Tested

Using both databases together has never been tested end-to-end.

4

No Plan for Old Data

No system automatically archives or deletes old traffic data.

5

Not Ready for AI

Neither database easily connects with modern AI tools.

The Core Trade-Off

A smart traffic system needs to do three things at once: handle huge amounts of data, answer location questions fast, and stay online everywhere. No single database does all three well.

MongoDB

Good at: flexible data and instant updates.

Struggles with: slows down when too many devices write data at once.

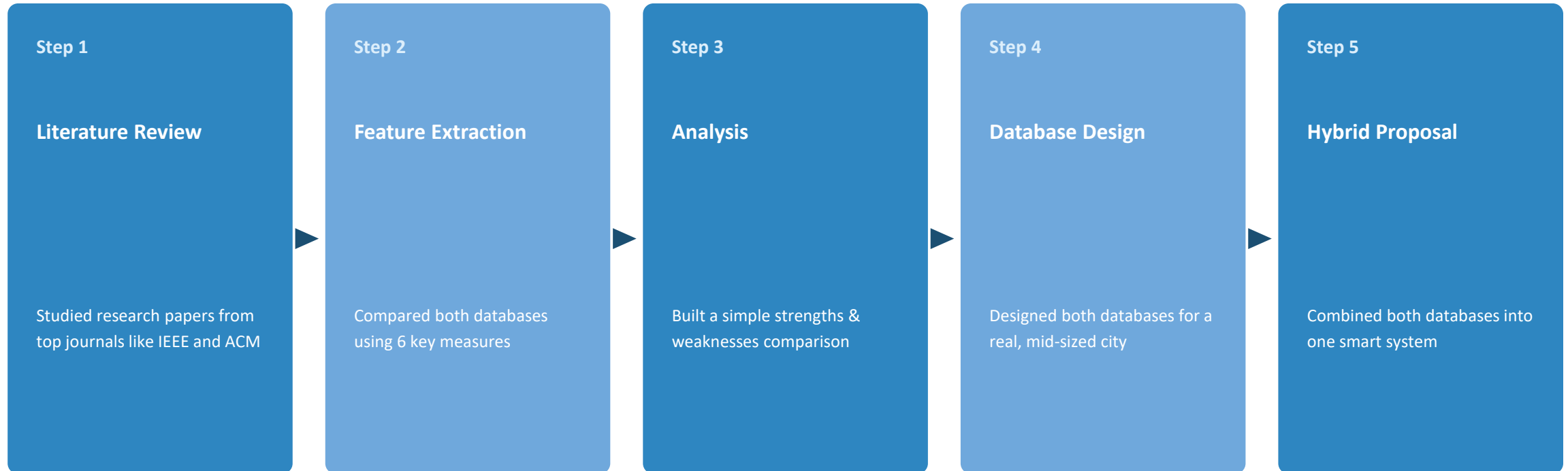
Apache Cassandra

Good at: never goes down, spreads data across many places.

Struggles with: can't guarantee perfect accuracy for safety-critical tasks.

Questions we asked: 1) Which database works best for which task? 2) How does each handle safety and accuracy? 3) Can we combine both without making things too complex?

Our 5-Step Process



From 247 papers, to 15 studies, to a 6-point comparison, to real database designs, to one combined solution.

How MongoDB Is Set Up

Test city: 2,000 intersections, 10,000 sensors, and 500,000 connected cars (about 100 TB of data per day).

Data Group	Split By	Fast Lookups For	Usage
Intersections	Intersection ID	Location, status	Read-heavy
Vehicle Events	Vehicle ID	Location, TTL, time	Write-heavy
Signal States	Intersection ID	Time, TTL	Mixed read-write
Incidents	Incident ID	Location, keyword, priority	Read-heavy, low write

A sample record (simplified)

```
{
  "intersection_id": "INT-0042",
  "location": { "type": "Point",
    "coordinates": [77.59, 12.97] },
  "signal_phases": [ ... ],
  "sensors": ["S-101","S-102"],
  "status": "active"
}
```

Why this works

- Splitting data by city zone avoids slowdowns during rush hour
- Old vehicle records auto-delete after 30 days
- Finds the nearest car or intersection in under a second

How Cassandra Is Set Up

Cassandra needs tables built around specific questions you'll ask, instead of flexible joins.

Table	Split By	Sorted By	Storage Method	Data Type
Sensor Readings	Sensor ID	Time (newest first)	24-hour batches	Raw sensor data
Vehicle Tracks	Vehicle ID	Time (newest first)	1-hour batches	GPS traces
Intersection Traffic	Intersection, date	Time (newest first)	Standard	Traffic counts
Signal Phase History	Intersection ID	Time (newest first)	1-hour batches	Signal changes
Congestion Alerts	Intersection, date	Alert time	Standard	Congestion events

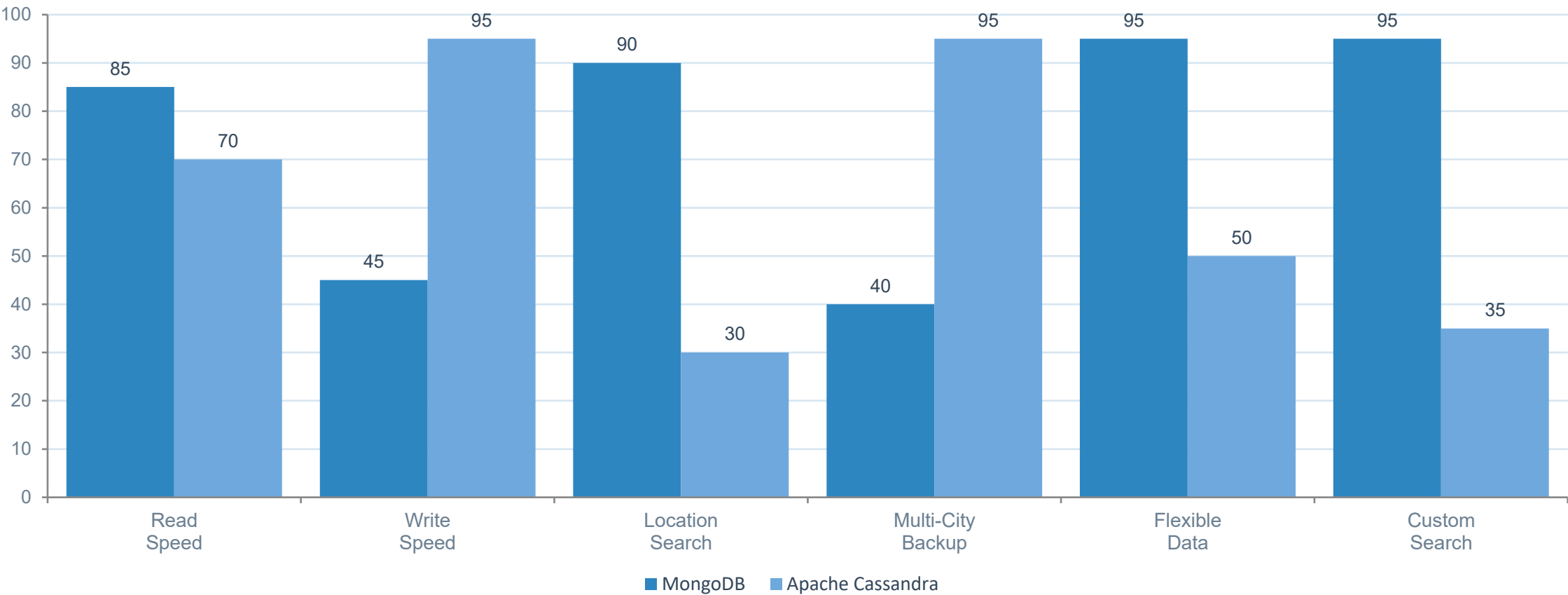
Why this works

- Handles about 500,000 updates every second at peak traffic times
- Groups data by time to save space and keep things fast
- Extra safety checks are used only for traffic-light coordination, so everything else stays quick

COMPARATIVE RESULTS

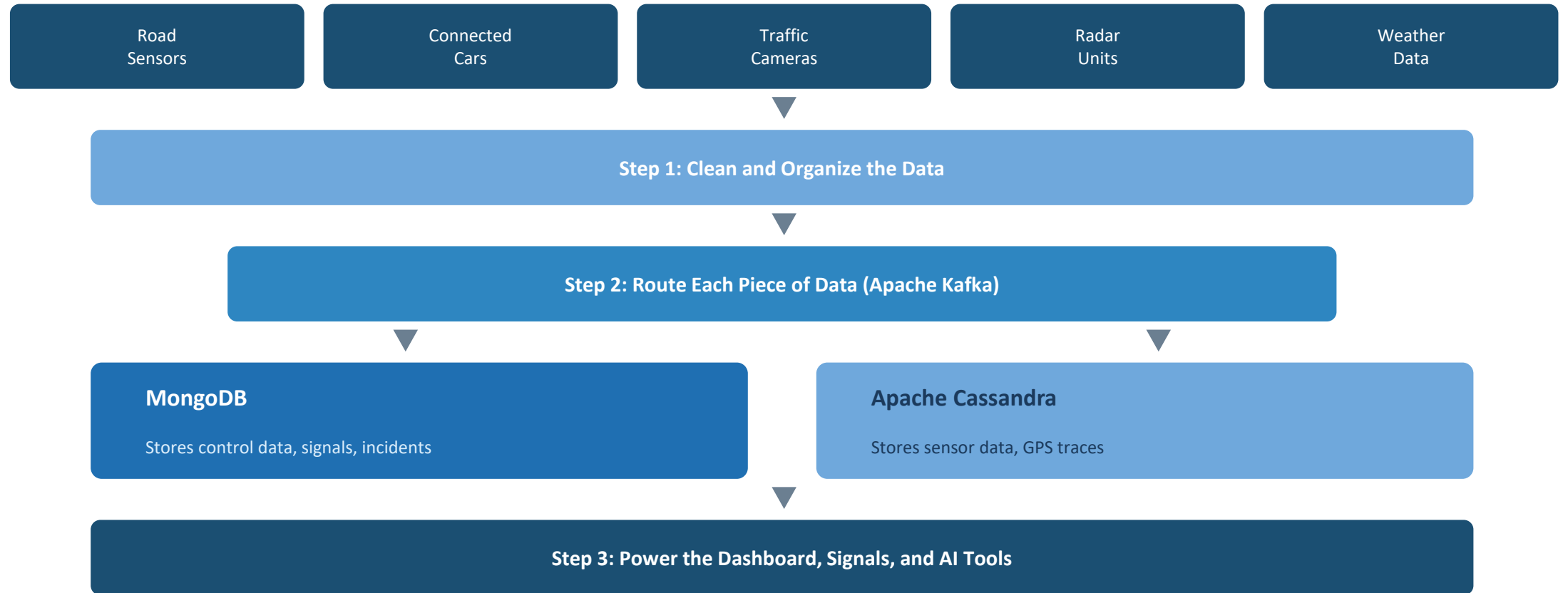
MongoDB vs. Cassandra, Side by Side

Scores from 0 to 100, based on results published in earlier research, across six important measures.



Using Both Databases Together

Send each type of data to the database that handles it best. This can cut costs by about 35%.



What Comes Next

1

AI-Ready Databases

Let databases work directly with AI models, without extra tools in between.

2

Databases at the Edge

Run small, lightweight databases right at traffic lights for instant response.

3

A Standard Way to Test

Build fair, realistic tests made just for traffic data.

4

Auto Data Cleanup

Automatically move or delete old data to save space and cost.

5

Safe Data Across Both Systems

Make sure updates in both databases always match up correctly.

6

A Digital Twin of the City

A live virtual copy of the city, used to test new ideas safely.

CONCLUSION

Two Tools, Two Jobs — Both Needed

- MongoDB is best for flexible data, location search, and safety-critical control.
- Cassandra is best for huge, fast-growing data spread across many places.
- No single database can do everything a smart city needs.
- Using both together cuts costs by about 35% and fixes each one's biggest weakness.

Pick the right tool for the job — not just the most popular one.

Thank You

Questions & Discussion

Tanisha Patel

Tanishapatel1805@gmail.com

Guided by Dr. Hemangini Patel — Vanita Vishram Women's University